

StandardScaler#

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Description:

Standardize features by removing the mean and scaling to unit variance.

The standard score of a sample x is calculated as:
$$z = \frac{x - u}{s}$$
where u is the mean of the training samples or zero if `with_mean=False`, and s is the standard deviation of the training samples or one if `with_std=False`. Centering and scaling happen independently on each feature by computing the relevant statistics on the samples in the training set. Mean and standard deviation are then stored to be used on later data using `transform`.

Function Signature

```
class sklearn.preprocessing.StandardScaler(*, copy=True,
with_mean=True, with_std=True)[source]#
```

Parameters:

copybool, default=True

with_meanbool, default=True

with_stdbool, default=True

```
class sklearn.preprocessing.StandardScaler(*, copy=True,
with_mean=True, with_std=True)[source]#
```

Parameters:

copybool, default=True

with_meanbool, default=True

with_stdbool, default=True

```
fit(X, y=None, sample_weight=None)[source]#
```

Parameters:

X{array-like, sparse matrix} of shape (n_samples, n_features)

sample_weightarray-like of shape (n_samples,), default=None

Parameters

`X{array}`

The data used to compute the mean and standard deviation used for later scaling along the features axis.

`sample_weightarray`

Individual weights for each sample. Added in version 0.24: parameter `sample_weight` support to `StandardScaler`.

`Xarray`

Input samples.

`yarray`

Target values (None for unsupervised transformations).

`**fit_paramsdict`

Additional fit parameters.

`input_featuresarray`

Input features. If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`. If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

`paramsdict`

Parameter names mapped to their values.

`X{array}`

The data used to scale along the features axis.

`X{array}`

The data used to compute the mean and standard deviation used for later scaling along the features axis.

`sample_weightarray`

Individual weights for each sample. Added in version 0.24: parameter `sample_weight` support to `StandardScaler`.

****paramsdict**

Estimator parameters.

X{array}

The data used to scale along the features axis.

Code Examples

$$z = (x - u) / s$$

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> print(scaler.fit(data))
StandardScaler()
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

Notes & Warnings

NOTE

See also `scaleEquivalent` function without the estimator API. PCA further removes the linear correlation across features with `'whiten=True'`.

Detailed Documentation

Documentation

Standardize features by removing the mean and scaling to unit variance.

The standard score of a sample x is calculated as:

where u is the mean of the training samples or zero if `with_mean=False`, and s is the standard deviation of the training samples or one if `with_std=False`.

Centering and scaling happen independently on each feature by computing the relevant statistics on the samples in the training set. Mean and standard deviation are then stored to be used on later data using `transform`.

Standardization of a dataset is a common requirement for many machine learning estimators: they might behave badly if the individual features do not more or less look like standard normally distributed data (e.g. Gaussian with 0 mean and unit variance).

For instance many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the L1 and L2 regularizers of linear models) assume that all features are centered around 0 and have variance in the same

order. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.

StandardScaler is sensitive to outliers, and the features may scale differently from each other in the presence of outliers. For an example visualization, refer to [Compare StandardScaler with other scalers](#).

This scaler can also be applied to sparse CSR or CSC matrices by passing `with_mean=False` to avoid breaking the sparsity structure of the data.

Read more in the [User Guide](#).

If `False`, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.

If `True`, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.

If `True`, scale the data to unit variance (or equivalently, unit standard deviation).

Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using `np.sqrt(var_)`. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. `scale_` is equal to `None` when `with_std=False`.

Added in version 0.17: `scale_`

The mean value for each feature in the training set. Equal to `None` when `with_mean=False` and `with_std=False`.

The variance for each feature in the training set. Used to compute `scale_`. Equal to `None` when `with_mean=False` and `with_std=False`.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.

The number of samples processed by the estimator for each feature. If there are no missing samples, the `n_samples_seen` will be an integer, otherwise it will be an array of dtype `int`. If `sample_weights` are used it will be a float (if no missing data) or an array of dtype `float` that sums the weights seen so far. Will be reset on new calls to `fit`, but increments across `partial_fit` calls.

Equivalent function without the estimator API.

Further removes the linear correlation across features with `'whiten=True'`.

NaNs are treated as missing values: disregarded in fit, and maintained in transform.

We use a biased estimator for the standard deviation, equivalent to `numpy.std(x, ddof=0)`. Note that the choice of `ddof` is unlikely to affect model performance.

Compute the mean and std to be used for later scaling.

The data used to compute the mean and standard deviation used for later scaling along the features axis.

Individual weights for each sample.

Added in version 0.24: parameter `sample_weight` support to `StandardScaler`.

Fit to data, then transform it.

Fits transformer to `X` and `y` with optional parameters `fit_params` and returns a transformed version of `X`.

Target values (None for unsupervised transformations).

Additional fit parameters.

Get output feature names for transformation.

If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`.

If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

Same as input features.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If `True`, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Scale back the data to the original representation.

The data used to scale along the features axis.

Copy the input X or not.

Online computation of mean and std on X for later scaling.

All of X is processed as a single batch. This is intended for cases when fit is not feasible due to very large number of `n_samples` or because X is read from a continuous stream.

The algorithm for incremental mean and std is given in Equation 1.5a,b in Chan, Tony F., Gene H. Golub, and Randall J. LeVeque. "Algorithms for computing the sample variance: Analysis and recommendations." *The American Statistician* 37.3 (1983): 242-247:

The data used to compute the mean and standard deviation used for later scaling along the features axis.

Individual weights for each sample.

Added in version 0.24: parameter `sample_weight` support to `StandardScaler`.

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to fit if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to fit.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in fit.

Configure whether metadata should be requested to be passed to the `inverse_transform` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to `inverse_transform` if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to `inverse_transform`.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for copy parameter in `inverse_transform`.

Set output container.

See [Introducing the set_output API](#) for an example on how to use the API.

Configure output of `transform` and `fit_transform`.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible

to update each component of a nested object.

Estimator parameters.

Configure whether metadata should be requested to be passed to the `partial_fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to `partial_fit` if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to `partial_fit`.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for `sample_weight` parameter in `partial_fit`.

Configure whether metadata should be requested to be passed to the `transform` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to transform if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to transform.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for copy parameter in transform.

Perform standardization by centering and scaling.

The data used to scale along the features axis.

Copy the input X or not.

Faces recognition example using eigenfaces and SVMs

Classifier comparison

Comparing different clustering algorithms on toy datasets

Demo of DBSCAN clustering algorithm

Demo of HDBSCAN clustering algorithm

A demo of K-Means clustering on the handwritten digits data

Comparing different hierarchical linkage methods on toy datasets

Column Transformer with Mixed Types

Pipelining: chaining a PCA and a logistic regression

Principal Component Regression vs Partial Least Squares Regression

Factor Analysis (with rotation) to visualize patterns

Combine predictors using stacking

Visualizing the probabilistic predictions of a VotingClassifier

Model-based and sequential feature selection

Common pitfalls in the interpretation of coefficients of linear models

Comparing Linear Bayesian Regressors

Lasso model selection via information criteria

Lasso model selection: AIC-BIC / cross-validation

L1 Penalty and Sparsity in Logistic Regression

Regularization path of L1- Logistic Regression

Poisson regression and non-normal loss

MNIST classification using multinomial logistic + L1

Tweedie regression on insurance claims

Visualizations with Display Objects

Displaying estimators and complex pipelines

Evaluation of outlier detection estimators

Advanced Plotting With Partial Dependence

Introducing the s